



ROBOCUPJUNIOR RESCUE MAZE 2026

TEAM DESCRIPTION PAPER

Tuton

Abstract

Our robot is designed to efficiently navigate and complete tasks in the RoboCup Rescue Maze competition. Through experience gained in previous competitions, we found that a well-structured design and thorough project planning are essential for achieving high performance and reliability.

To address the challenges of uneven terrain, we developed a custom articulated chassis equipped with deformable silicone tires, enabling stable traversal of bumps, ramps, and stairs. The robot employs a distributed architecture consisting of multiple STM32 microcontrollers and a Raspberry Pi, improving modularity, maintainability, and stable electronic operation. LiDAR-based mapping and an orientation-aware Dijkstra's algorithm enable efficient autonomous exploration.

The victim detection system, refined through extensive development and testing, ensures reliable victim identification and rescue kit deployment. Through rigorous performance evaluation, we validated the robot's capabilities under competition-like conditions and confirmed compliance with competition requirements. Our team remains committed to continuous improvement and innovation, striving for excellence in the RoboCup Rescue Maze competition.

1. Introduction

a. Team

Shuji Igarashi: He has been participating in RoboCupJunior since 2023. As Captain, he led project management and system integration. Also, he designed all the PCBs and contributed to software development.

Ryouta Koudu: He has been participating in RoboCupJunior since 2023. He was responsible for mechanical design and production. Particularly, he devoted his efforts to designing a mobility system that could maintain stable movement even on the challenging terrain in the Dangerous Zone.

Shuichiro Mitsumoto: He has been participating in RoboCupJunior since 2025. He was responsible for overseeing overall software development and leading software development and implementation. In particular, he focused on improving the accuracy of victim detection through image processing.

2. Project Planning

a. Overall Project Plan

Objective: Our objective is to win first place at the RoboCupJunior World Championship. To achieve this, we decided to focus on reliability and robustness of our robot. There are many tasks to overcome in the Rescue Maze League, so we have listed the requirements and decided on the solutions for each task. To secure enough reliability, we sometimes adopted multiple solutions for one objective.

Table 1: Overview of Requirements and Final Solutions

Requirement	Possible Solutions (Underlined: Final solution)	Final Solution and Rationale
Move precisely to the next tile	<u>Time-based control</u> , <u>Distance Sensor</u>	To improve positioning accuracy, the robot uses distance sensors to travel 30 cm when a wall is present in front of or behind the robot. Otherwise, time-based control is used.
Turn 90° precisely	Time-based control, <u>Gyroscope-based control</u>	We adopted gyroscope-based control using the BNO085 IMU, enabling accurate turns even when the wheels become stuck.
Traverse bumps, stairs, and ramps	<u>Silicone wheels</u> , Crawler, Suspension, <u>Articulated structure</u>	We developed custom deformable silicone wheels and an articulated structure that provide high traction and reliable traversal of bumps, stairs, and ramps.

Deploy Rescue Kits	Micro servo , Push Solenoid	We adopted a micro servo-based mechanism because it is compact and space-efficient.
Detect Walls	Ultrasonic sensor, ToF Sensor , LiDAR , Microswitch	To achieve high accuracy, rich environmental data, and fast response, we combined VL53L4CX ToF sensors with a T-mini Plus LiDAR.
Detect Obstacles	Loadcell, Microswitch, Tactile Switch	Experimental evaluation showed that tactile switches provide the best balance of responsiveness and durability.
Detect Victims	UnitV Cam , OpenMV Cam, Raspberry Pi Cam	Early testing demonstrated that the UnitV Cam provides sufficient processing capability for victim detection while remaining compact and easy to integrate.
Detect Color Tiles	Color Sensor , Reflectance sensor , Camera	Color sensors alone cannot reliably distinguish black tiles when the robot is lifted by bumps, while cameras struggle to differentiate black and silver tiles. Therefore, both sensing methods are used.

Project Schedule: Since we had participated in the Rescue Line sub-league until 2025 season, development of our Rescue Maze robot began in August 2025. To maximize progress within the limited development period, we meticulously planned our schedule and focused on rapid prototyping and continuous improvement. Our development timeline is shown in [Table 2](#).

Table 2: Overview of Project Schedule

Milestone	Detail, Definition of Done	Assigned to	Deadline	Actual Finish Date
Project Started: 2025-08-18				
Brainstorm ideas and finalize the conceptual design of the 1 st prototype	- Finish part selection - Determine part location	Shuji, Ryouta, Shuichiro	10 Oct 2025	18 Oct 2025
Define the software architecture	- Define the roles of each processor - Define file/class architecture in Raspberry Pi - Decide the software task allocation among members	Shuji, Shuichiro	18 Nov 2025	18 Nov 2025
Develop an efficient Exploration Algorithm	Implement a search algorithm on a laptop that is faster than the extended right-hand rule	Shuji	15 Nov 2025	14 Nov 2025
Develop Victim Finding Algorithm using UnitV Cam	Achieve a detection rate of 80% or higher	Shuichiro	15 Dec 2025	20 Dec 2025
Complete the design and the production of the 1st prototype hardware	Finish PCB production	Shuji	10 Nov 2025	6 Nov 2025
	Finish chassis production	Ryouta	10 Nov 2025	6 Nov 2025
	Finish assembling (=CheckPoint1: Completion of 1st prototype)	Shuji, Ryouta	12 Nov 2025	7 Nov 2025
Reflect on the deficiency of the 1 st prototype and come up with idea of the 2 nd prototype	- Finished part selection - Determined part location	Shuji, Ryouta, Shuichiro	15 Nov 2025	8 Nov 2025

Complete the design and the production of the 2 nd prototype hardware	Finish PCB production	Shuji	20 Dec 2025	15 Dec 2025
	Finish chassis production	Ryouta		
	Finish assembling	Ryouta		
Software implementation on 1 st prototype	Run a simple program that performs maze exploration while rescuing victims	Shuichiro	20 Dec 2025	15 Dec 2025
Software development on 2 nd prototype	Implement all software features for the competition	Shuichiro	6 Jan 2026	11 Jan 2026
CheckPoint2: Regional competition 13 Jan 2026				
Reflect on the deficiency of the 2 nd prototype and come up with idea of the 3 rd prototype	- Determine the design changes for the new model - Establish the development schedule for it	Shuji, Ryouta, Shuichiro	15 Jan 2026	15 Jan 2026
Complete the design and the production of the 3 rd prototype hardware	Finish PCB production	Shuji	1 Mar 2026	8 Mar 2026
	Finish chassis production	Ryouta	1 Mar 2026	5 Mar 2026
	Finish assembling	Ryouta	15 Mar 2026	16 Mar 2026
Software development on 3 rd prototype	Implement all software features on the new robot	Shuichiro	20 Mar 2026	27 Mar 2026
CheckPoint3: Japan Open 28,29 Mar 2026				
Reflect on the deficiency of the 3 rd prototype and come up with idea of the final robot	- Determine the design changes for the new model - Establish the development schedule for it	Shuji, Ryouta, Shuichiro	5 Apr 2026	3 Apr 2026
Massive refactoring of 3 rd prototype software	Refactor the Raspberry Pi code to improve future developability	Shuji	30 Apr 2026	4 May 2026
Develop cognitive target finding algorithm	Achieve a detection rate of 80%	Shuichiro	31 May 2026	10 May 2026
	Achieve a detection rate of 90%	Shuichiro	15 Jun 2026	9 Jun 2026
Designing an algorithm that supports fields with multi-layered crossings	Enable accurate mapping even when connected by slopes with different inclinations	Shuji	31 May 2026	26 May 2026
Complete the design and the production of the final robot hardware	Finish chassis production	Ryouta	15 May 2026	4 May 2026
	Finish new wheels production	Ryouta	31 May 2026	9 Jun 2026
	Complete final prototype hardware production	Ryouta	5 Jun 2026	11 Jun 2026
World Championship 1 - 6 Jul 2026				

We defined checkpoints to mark the completion of each prototype. At each checkpoint, all team members evaluated the weaknesses of the current robot and developed a clearer vision for the next prototype.

To accelerate development, we built two robots, allowing hardware and software development to proceed in parallel (*Figure 1*). Team members independently explored solutions to challenges, enabling fast iteration. Key software components, including victim detection and autonomous exploration algorithms, were also developed and tested on laptops before hardware completion, reducing later integration work.

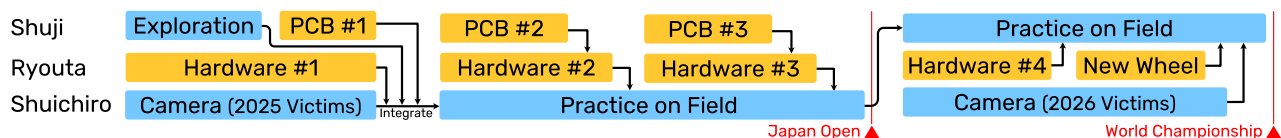


Figure 1: Parallel Development

b. Integration Plan

To facilitate both component-level development and system integration, the robot was designed with a modular architecture that allows individual components to be easily removed, replaced, and tested independently.

The hardware unit structure of the robot is shown in *Figure 2*. Each module can be accessed and detached individually, improving maintainability, simplifying debugging, and enabling efficient hardware iteration during development.

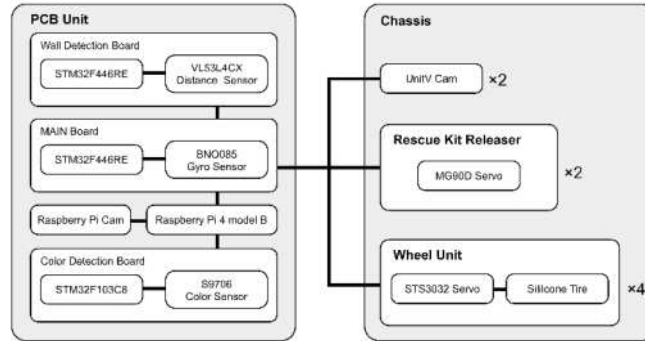


Figure 2: Integration Plan

Details of the electrical connections and communication interfaces between components are described in *3. Hardware - Electronic Design and Manufacturing*. Details of software integration are described in *4. Software - General Software Architecture*. How each component satisfies the requirements of this competition can be found in *Table 1*.

3. Hardware

Overview: Most of the robot's mechanical components were manufactured using Flashforge Adventurer 5M and ELEGOO Neptune 3 3D printers with PLA filaments. This approach enabled rapid design iteration throughout development and allowed efficient utilization of the limited space available inside the robot.

To improve stability on ramps, the robot was designed with a low-profile structure, keeping its height below 15 cm while maximizing the wheelbase length. In addition, custom wheels with large, deformable tires were developed to enhance terrain adaptability. This design provides sufficient traction and shock absorption to reliably traverse the 20 mm round bumps used in the competition field.

The robot is divided into two primary modules: the chassis and wheel assembly, and the electronics unit (*Figure 3*). The electronics unit contains the major electronic components and can be removed from the robot as a single assembly for maintenance and troubleshooting.

The chassis and wheel assembly were designed and manufactured by Ryouta, while the electronics unit was designed and assembled by Shuji. This modular architecture enabled parallel development by team members and improved both maintainability and system integration efficiency.



Figure 3: The Robot Designed in CAD

a. Mechanical Design and Manufacturing

Chassis & Wheels: The robot chassis is built on a CNC-machined aluminum base plate. This structure provides high rigidity and reliable support for the entire robot while lowering the center of gravity, resulting in improved driving stability.

To enhance terrain adaptability, the robot body is divided into front and rear sections connected by a central pivot mechanism (*Figure 4*). This articulated structure allows the two sections to twist relative to each other, ensuring that all four wheels maintain contact with the ground even when traversing uneven surfaces. As a result, the robot can drive stably over bumps, ramps and stairs.



Figure 4: Articulated Structure

Each wheel consists of three main components: a PLA hub, a deformable TPU suspension structure, and a silicone-molded tread. The combination of the high grip provided by the silicone tread and the shock-absorbing characteristics of the TPU structure (*Figure 5*) significantly improves mobility on bumps, slopes, and other challenging terrain.

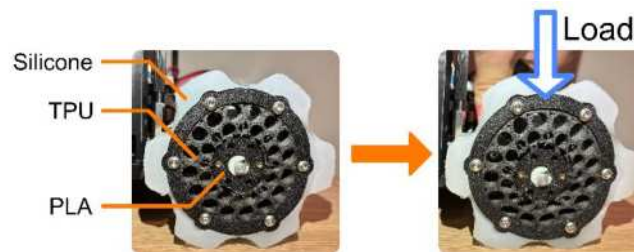


Figure 5: Deformable Wheel

The silicone tread features a contoured profile designed to conform to the 20 mm round bump used in the competition field, maximizing traction and bump-crossing performance. In addition, the suspension characteristics of the TPU structure can be tuned by modifying its geometry. Multiple designs were prototyped and experimentally evaluated, and the most effective configuration was selected for the final robot.

Rescue kit deployment mechanisms: The rescue kit is an 11 mm cubic capsule manufactured using a 3D printer. A freely moving nut is enclosed inside the cube to absorb impact energy when the kit contacts the floor or walls (*Figure 6*). This design significantly reduces bouncing after deployment, allowing the rescue kit to remain close to the intended target location and improving delivery reliability.



Figure 6: Rescue Kits with Nuts

The robot carries a total of eight rescue kits. Four kits are stacked vertically on each side of the robot and are fed individually into the deployment mechanism (*Figure 7*). An MG90D servo motor pushes the kits out one at a time, enabling controlled and reliable deployment. The rescue kits are deployed from the sides of the robot rather than from inside to directly underneath. This configuration prevents the robot from accidentally driving over or displacing a deployed rescue kit. By minimizing interference between the robot and the rescue kit after deployment, the system improves placement accuracy and overall rescue performance.



Figure 7: Rescue Kit Releaser

b. Electronic Design and Manufacturing

The electronic architecture of the robot is shown in *Figure 8*.

Custom PCBs were designed and manufactured for each functional subsystem, resulting in compact wiring and efficient use of space. Extensive use of surface-mount components further reduced the overall footprint of the electronics.

Rather than connecting sensors and actuators directly to a central processor, each board incorporates its own STM32 microcontroller and communicates with other modules via UART. This distributed architecture simplifies component-level development and testing while reducing the risk of communication failures caused by shared I²C buses.

The Main Board, Wall Detection Board, Raspberry Pi, and Color Detection Board are integrated into a single PCB Unit. These modules are arranged to maximize space efficiency and can be easily detached from the chassis for maintenance and development.

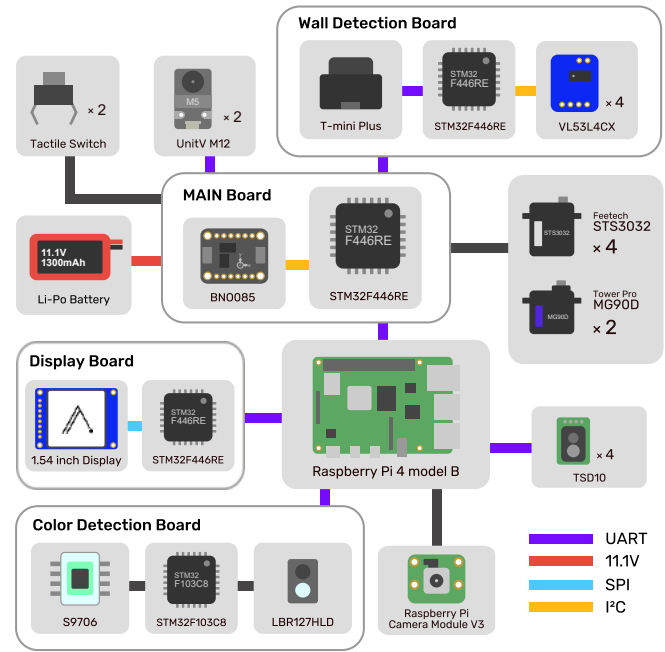


Figure 8: Electronic Parts Diagram

The power distribution system is illustrated in *Figure 9*.

The robot is powered by an 11.1 V, 1300 mAh Li-Po battery. Dedicated DC-DC converters are used for individual actuator systems, ensuring stable power delivery while minimizing the impact of electrical faults on other subsystems.

Each PCB was tested independently before integration into the robot. During development, servo motor noise occasionally affected microcontroller stability. This issue was investigated using an oscilloscope and resolved through improvements to the power filtering circuitry and PCB layout.

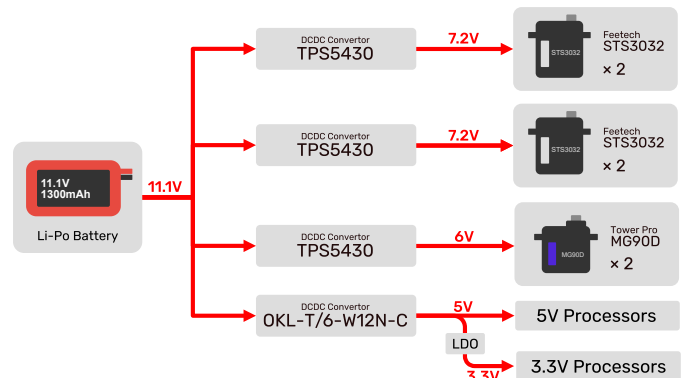


Figure 9: Power Supply

4. Software

Overview: Software was developed for the Raspberry Pi, STM32 microcontrollers, and UnitV camera modules. Software development was carried out by Shuji and Shuichiro. To facilitate collaboration, the source code was managed through GitHub, and key classes and functions were documented, enabling efficient development and maintenance.

In addition to testing on the physical robot, a custom simulation environment was developed to support software design and evaluation. This environment was particularly useful for the development and optimization of the maze exploration algorithm.

a. General software architecture

Architecture: The robot employs a distributed processing architecture consisting of a Raspberry Pi 4 Model B, STM32F446RE microcontrollers, an STM32F103C8 microcontroller, and UnitV camera modules. Software is developed in Python 3.13.5 for the Raspberry Pi and on the UnitV running firmware v0.6.2, while the STM32 microcontrollers are programmed in C++. The system architecture is illustrated in [Figure 10](#).

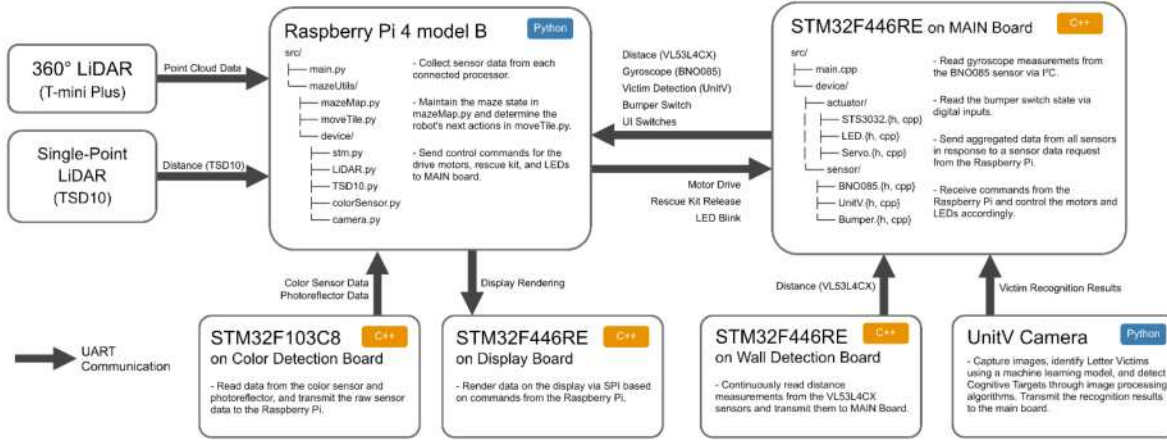


Figure 10: Software Architecture

The Raspberry Pi serves as the main processor and executes the core logic, including mapping, navigation, and maze exploration. The STM32 microcontrollers operate as sub-processors responsible for sensor acquisition and actuator control. Victim detection is performed exclusively on the UnitV camera module, which processes camera images and reports detection results to the main processor.

The software running on the Raspberry Pi is organized into two layers: a hardware abstraction layer that manages sensors and actuators, and a higher-level layer that implements the exploration and decision-making algorithms.

Algorithm Overview: The flowchart in [Figure 11](#) shows the overall software flow.

The robot explores the maze one tile at a time while continuously updating its internal map based on detected walls. When a checkpoint is reached, the current map and exploration state are saved. In the event of a Lack of Progress, the robot restores the most recent checkpoint data and resumes operation. During each tile transition, the robot simultaneously performs black tile detection, victim recognition, and obstacle detection, executing the appropriate response whenever one of these elements is identified.

Maze exploration is performed using a custom algorithm that prioritizes visiting unexplored tiles. The details of this algorithm are described in [b. Innovative solutions](#). After all reachable tiles have been explored, the robot calculates the shortest path to the start tile and returns to complete the run.

Communication: Communication between processors is implemented using UART with a master–slave architecture. All communication is initiated by the main processor using a request–response protocol. When sensor data is required, the main processor sends a request packet and receives a response containing the requested data. For control

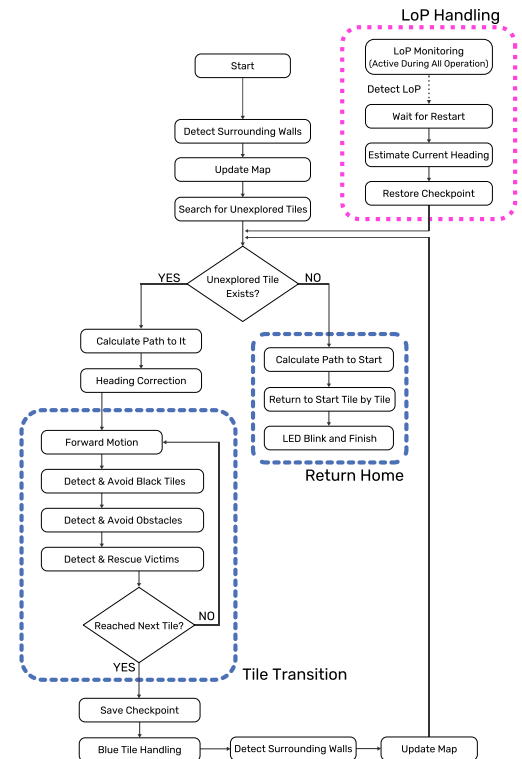


Figure 11: Overall Software Flowchart

commands, the sub-processor returns an acknowledgment (ACK) packet after successful processing. This approach reduces unnecessary communication while ensuring reliable data exchange.

Each packet consists of a *Type* field, a *Sequence Number*, a *Payload*, and a *Check Digit* (Figure 12). The *Type* field identifies the purpose of the packet, while the *Sequence Number* is used to match requests and responses. To detect transmission errors, the *Check Digit* is calculated as the XOR of all preceding packet bytes. Packets that fail validation are discarded.



Figure 12: Communication Protocol

Victim Detection: Victim detection is consistently performed on the UnitV. The UnitV captures 112x112, RGB565 images and uses its built-in K210 chip to detect letter victims and cognitive targets in parallel.

Detection of letter victims: First, the captured image is converted to grayscale and binarized using a threshold. This allows the system to handle environmental changes by simply adjusting the threshold. The processed image is then passed through a trained neural network model based on the MobileNetV1 architecture with a width multiplier of $\alpha=0.75$ because its structure matches the hardware constraints of the K210 chip used in the UnitV. To improve robustness, focal loss, data augmentation (CutMix, jittering, and flipping), and Out-of-Distribution (OoD) detection using the Maximum Logit Score were applied during training and inference. Experimental results showed that the threshold avoided approximately 40% (73/185) of false detections on fake victims while the precision decreased from 93.7% to 93.1%.

Detection of cognitive targets: Candidates for cognitive targets are first detected from the image using a flood-fill method. Shape-based filtering is then applied, and the largest valid candidate is selected. After estimating the radius and center of the selected candidate, 3x3 pixels square regions are sampled in 12 directions, centered at points whose distances from the center are 0.1, 0.3, 0.5, 0.7, and 0.9 times the radius. The color of each layer and the total score are determined by voting based on these samples. After that, the target is considered detected only when the agreement level of these votes exceeds a threshold and the same score is detected across consecutive frames.

We adopted a rule-based approach instead of CNN-based methods because running multiple CNN models on the UnitV is impractical due to hardware constraints. In addition, the large appearance variations of cognitive targets and the need for environment-specific tuning make rule-based methods more suitable.

b. Innovative solutions

Path Planning Algorithm: The robot explores the maze using a frontier-based strategy. Whenever it enters a new tile, wall information is measured and incorporated into an internal map. Newly discovered neighboring tiles are registered as unexplored tiles. The robot then repeatedly selects the nearest unexplored tile and navigates to it using the lowest-cost path. After all reachable tiles have been explored, the robot calculates the shortest path back to the start tile and returns to complete the run.

Path planning is performed using Dijkstra's algorithm. Unlike conventional tile-based graph representations, each graph node includes both the tile position and the robot's orientation (Figure 13). This allows the planner to account for turning costs in addition to translational movement costs.

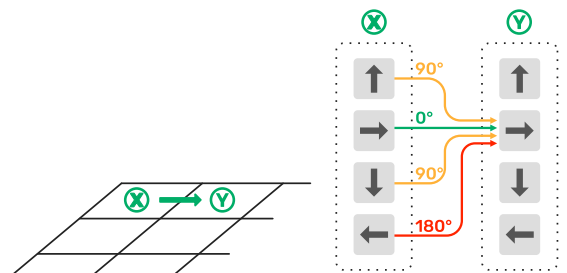


Figure 13: Cost Calculation Using Orientation-Aware Vertices

Moving forward, turning 90°, and turning 180° are assigned different costs based on experimentally measured execution times. By incorporating orientation into the graph, the calculated path more accurately reflects the robot's actual travel time and results in more efficient navigation.

To evaluate the effectiveness of the proposed exploration algorithm, a custom simulation environment was developed and utilized. The simulation framework and experimental results are described in [5. Performance evaluation](#) section.

Dynamic 3D Mapping: To accurately map multi-level environments, we developed a height-based layer matching system. New layers are generated dynamically whenever the robot discovers a new floor through a ramp. Vertical displacement is estimated using gyro data and traveled distance, allowing layers at similar heights to be matched and treated as a single layer.

This approach enables reliable mapping even in complex fields with multiple ramps of varying inclinations ([Figure 14](#)). To prevent mapping errors caused by stairs, the system distinguishes stairs from actual floor transitions and avoids creating unnecessary layers when no layer change has occurred.

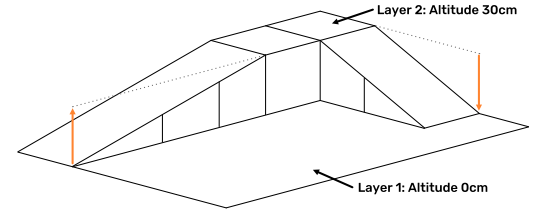


Figure 14: Layer matching based on height information

5. Performance evaluation

Hardware Capability: For each hardware improvement, extensive testing was conducted to verify its effectiveness and ensure system reliability. Quantitative metrics such as success rate and performance indicators were measured, and the results were used to guide subsequent design iterations and refinements. Representative examples of identified issues, implemented solutions, and evaluation methods are presented in [Table 3](#).

Table 3. Examples of Hardware Reliability Improvements and Their Evaluation

Problem	Solution	Evaluation Method	Result
Rescue kits bounced after deployment.	A freely moving nut was placed inside each rescue kit to absorb impact energy.	Each rescue kit was dropped 20 times from a height of 5 cm, and the average displacement from the target landing position was measured.	The average displacement decreased from 94.1mm to 23.0mm.
The robot could not reliably traverse a bump located on a ramp.	The wheel diameter was increased from 65 mm to 80 mm.	The time required to travel one tile on a 25° ramp with a 10 mm bump was measured 10 times, and the average value was calculated.	The average time decreased from 14.2 sec to 3.4 sec.
The robot became stuck while turning on a 20 mm bump.	An articulated structure was introduced between the front and rear sections of the robot.	The robot was commanded to perform a 180° turn on a 20 mm bump 20 times, and the stall rate was measured.	The stall rate decreased from 65% to 0%.

Exploration Algorithm Cost: To evaluate our exploration algorithm, we developed a simulation environment compatible with RCJ CMS map JSON data. Random maze fields resembling Rescue Maze environments were generated and used for benchmarking. Experimental results ([Figure 15](#)) showed that the proposed method consistently achieved lower exploration costs than the Extended Right-Hand Rule while also exhibiting lower cost variance across mazes of similar size.

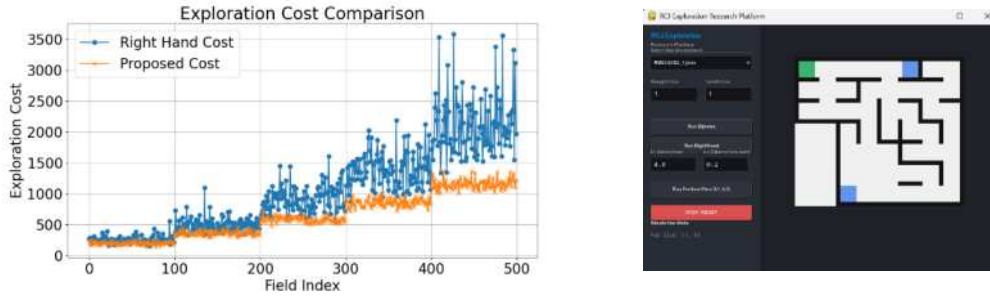


Figure 15. Evaluation of the Exploration Algorithm Using a Custom Simulation Environment

Letter Victim Detection Precision: To verify whether our detection was sufficiently accurate, we prepared a new validation dataset consisting of a total of 3,029 images. Using this dataset, we evaluated the detection accuracy of our trained model. In addition to the target classes omega, phi, and psi, we also tested two negative classes: other, consisting only of background images, and fake_victims, consisting of fake victims that include other Greek or alphabetic characters. The results are shown in [Table 4](#).

Table 4. Victim Detection Accuracy on the Validation Dataset

Class	Accuracy	Precision	Recall
Omega(Ω)	95%	95%	84%
Phi(Φ)	94%	95%	82%
Psi(Ψ)	90%	93%	79%
other	94%	97%	99%
fake_victims	83%	56%	87%

These results indicate that our model achieved sufficient accuracy for practical use in the competition environment, while also showing that fake victims remain the main target for further improvement.

Cognitive Target Detection Speed: To verify whether our detection speed was sufficiently fast, we measured the time required for each step of the detection process. Based on these results, we determined which steps should be optimized, enabling more effective implementation and providing quantitative performance metrics. [Table 5](#) presents an example of this process: we identified blob searching as the bottleneck and optimized it by modifying the firmware to process multiple thresholds simultaneously.

Table 5. An Example of Optimizing Detecting process

Metric	Before Optimization	After Optimization	Change
FPS	22.7 frames/s	29 frames/s	+31.8%
Avg. Loop Time	44,114 μ s	34,071 μ s	-22.8%
Avg. find_blobs() Time	20,425 μ s	9,640 μ s	-52.8%
Avg. Other Processing Time	23,689 μ s	24,431 μ s	+3.1%

6. Conclusion

After transitioning from Rescue Line, we successfully developed a robust, modular platform for the 2026 competition. We learned from early prototype failures in bump-crossing and sensor placement to create a more dependable final design. Our distributed processing architecture significantly improved system efficiency and reliability. We look forward to showcasing our technical growth and exchanging ideas at the World Championship.

Appendix (optional)

- Our website: <https://tuton-rcj.github.io/>
- Our YouTube Channel: https://www.youtube.com/@tuton_RCJ
- Our X Account: https://twitter.com/tuton_RCJ
- Our GitHub: <https://github.com/tuton-RCJ>

References

a. Software tools & Platforms

- [Visual Studio Code](#)
- [Platform IO](#)
- [GitHub](#)
- [Autodesk Fusion](#)
- [KiCAD](#)
- [Raspberry Pi OS](#)
- [MaixPy IDE](#)

b. Libraries

- [Numpy](#)
- [OpenCV-Python](#)

c. Algorithms

- [Dijkstra's method](#)
- [MobileNet V1](#)

d. Hardware

- [Raspberry Pi 4 model B](#)
- [STM32F446RE](#)
- [STM32F103C8](#)
- [Raspberry Pi Camera V3](#)
- [UnitV M12](#)
- [BNO085](#)
- [STS3032](#)
- [MG90D](#)
- [T-mini Plus](#)
- [VL53L4CX](#)
- [TSD10](#)
- [S9706](#)
- [Flashforge Adventurer5M](#)
- [ELEGOO Neptune 3](#)